



RISC-v数据通路设计

2025年秋

本讲概要

□ Risc-v指令格式（回顾）

□ 数据通路

□ 指令执行流程

- R型指令：add/sub
- I型指令：addi
- 访存I型指令：lw
- S型指令：sw
- B型指令：beq
- U型指令：lui/auipc
- J型指令：jal
- 跳转I型指令：jalr

RISC-V 的指令格式 (I)

- ❑ R-格式：寄存器指令，指定指令中的3个寄存器
- ❑ I-格式：指令中包含立即数，用于带一个常数的算术指令以及加载指令
- ❑ S-格式：store指令

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0				
funct7				rs2			rs1		funct3		rd			opcode		R-type		
imm[11:0]						rs1		funct3		rd			opcode		I-type			
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type		
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode		B-type
imm[31:12]										rd			opcode			U-type		
imm[20]		imm[10:1]				imm[11]		imm[19:12]			rd			opcode		J-type		

RISC-V 的指令格式 (II)

□ B-格式：分支指令

□ U-格式：大立即数

- 两个指令lui (load upper imm), auipc (add upper imm to PC)

□ J-格式：唯一指令jal

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
funct7				rs2			rs1		funct3		rd			opcode		R-type
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]	imm[11]	opcode		B-type
imm[31:12]										rd			opcode		U-type	
imm[20]		imm[10:1]				imm[11]		imm[19:12]			rd			opcode		J-type

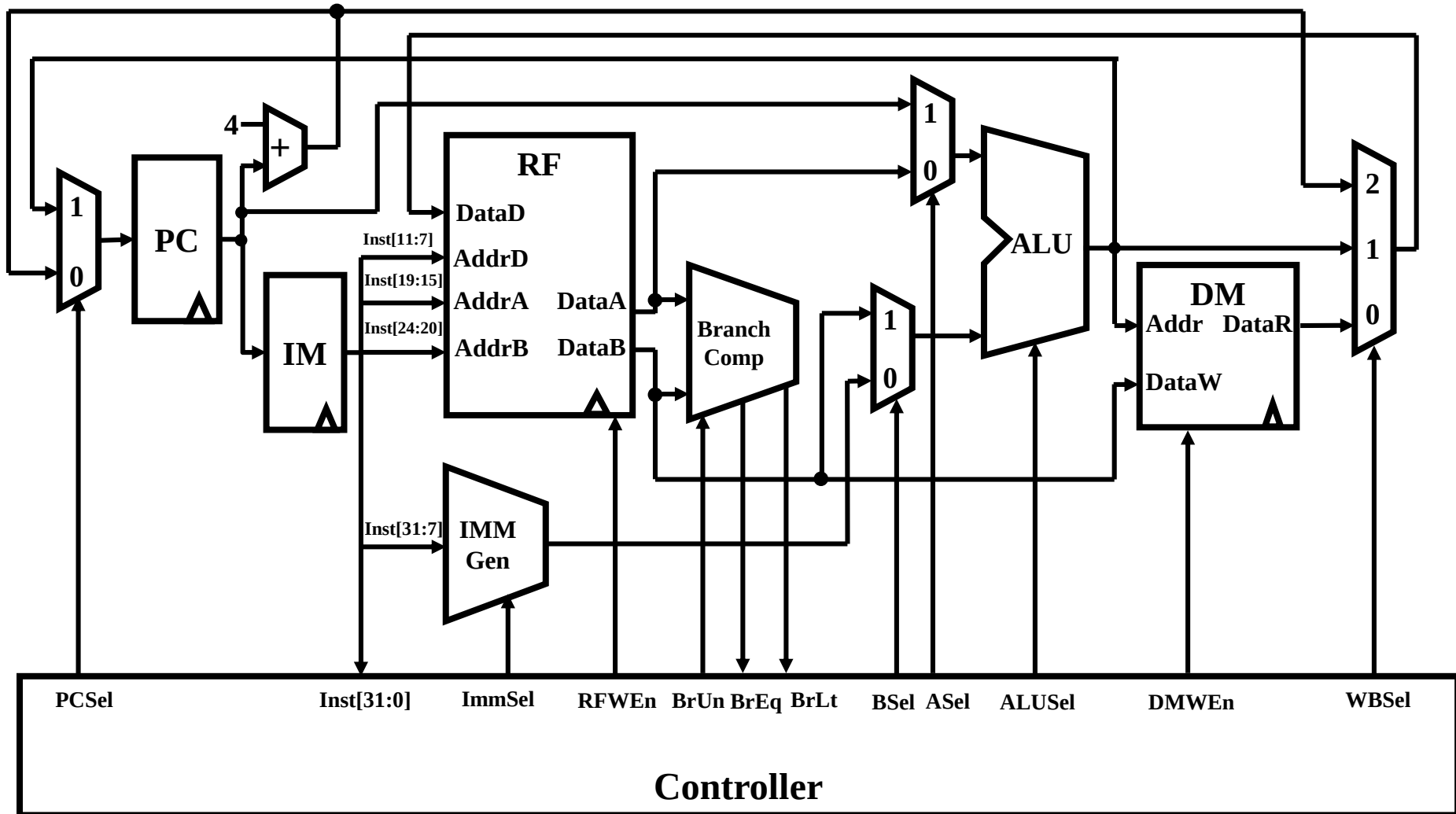
RISC-V 指令格式总结

- 代码与数据一样保存在程序地址空间非常重要，硬件和软件在一定程度上同等处理
- RISC-V 机器语言指令:

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
funct7				rs2			rs1		funct3		rd			opcode		R-type
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]	imm[11]	opcode	B-type	
imm[31:12]										rd			opcode		U-type	
imm[20]		imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type	

数据通路设计

RISC-V RV32I 单周期CPU



R型指令—— add, sub, etc.

add \ sub

add rd rs1 rs2

sub rd rs1 rs2

指令功能

$R[rd] = R[rs1] + R[rs2]$

$R[rd] = R[rs1] - R[rs2]$

执行过程

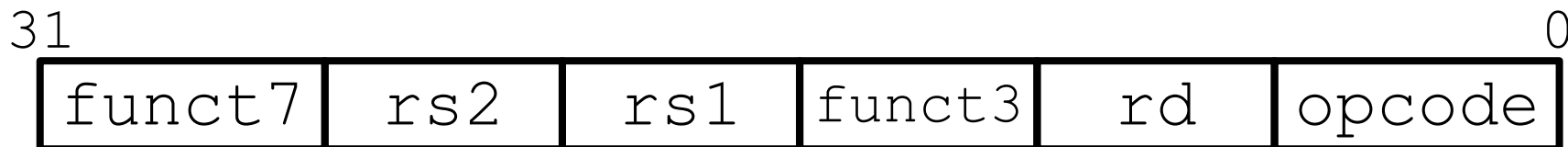
取指令

分析指令

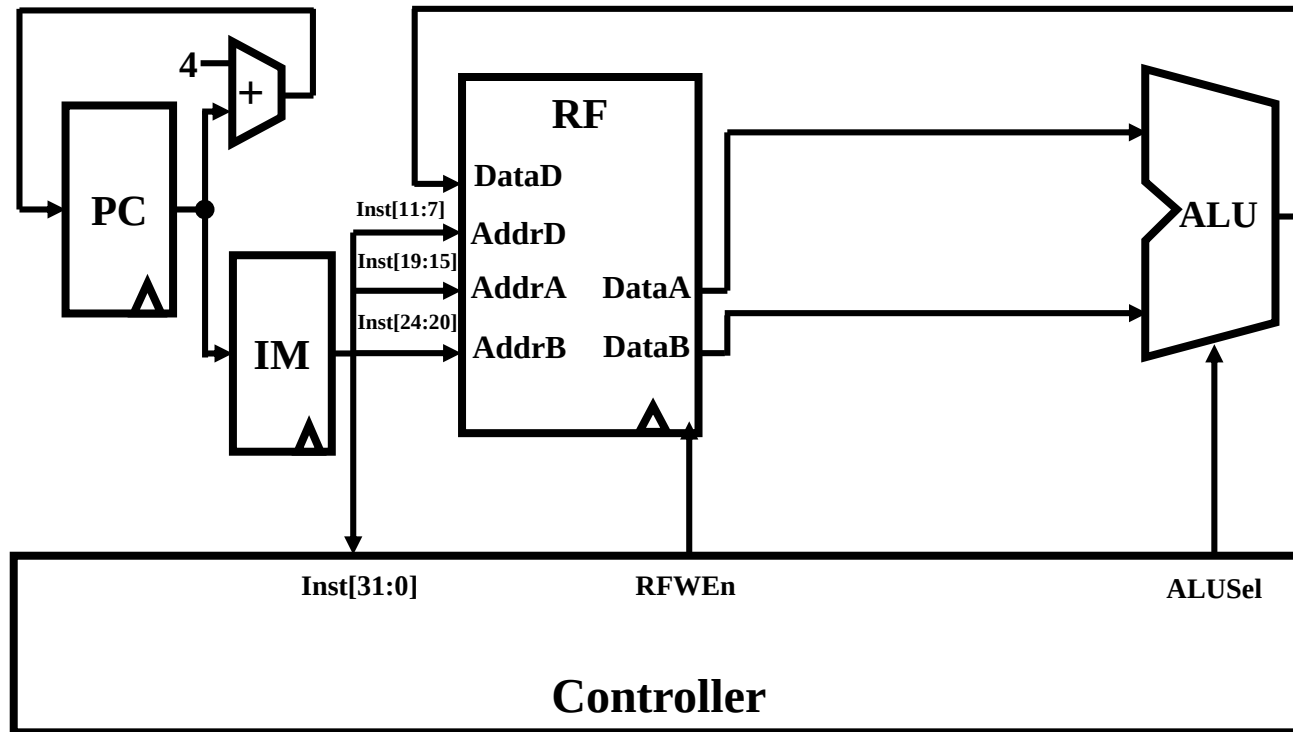
执行指令

- 取操作数
- 运算
- 结果写回

计算下一条指令的地址



数据通路设计1

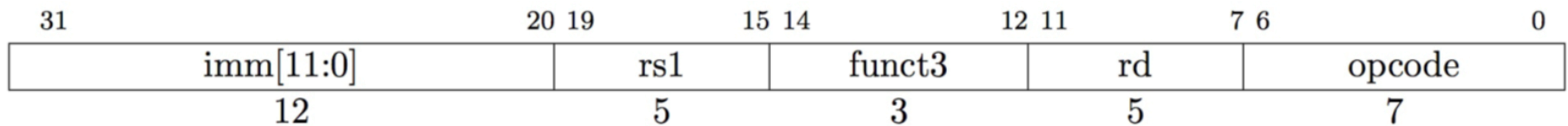


算术I型指令—— addi, etc.

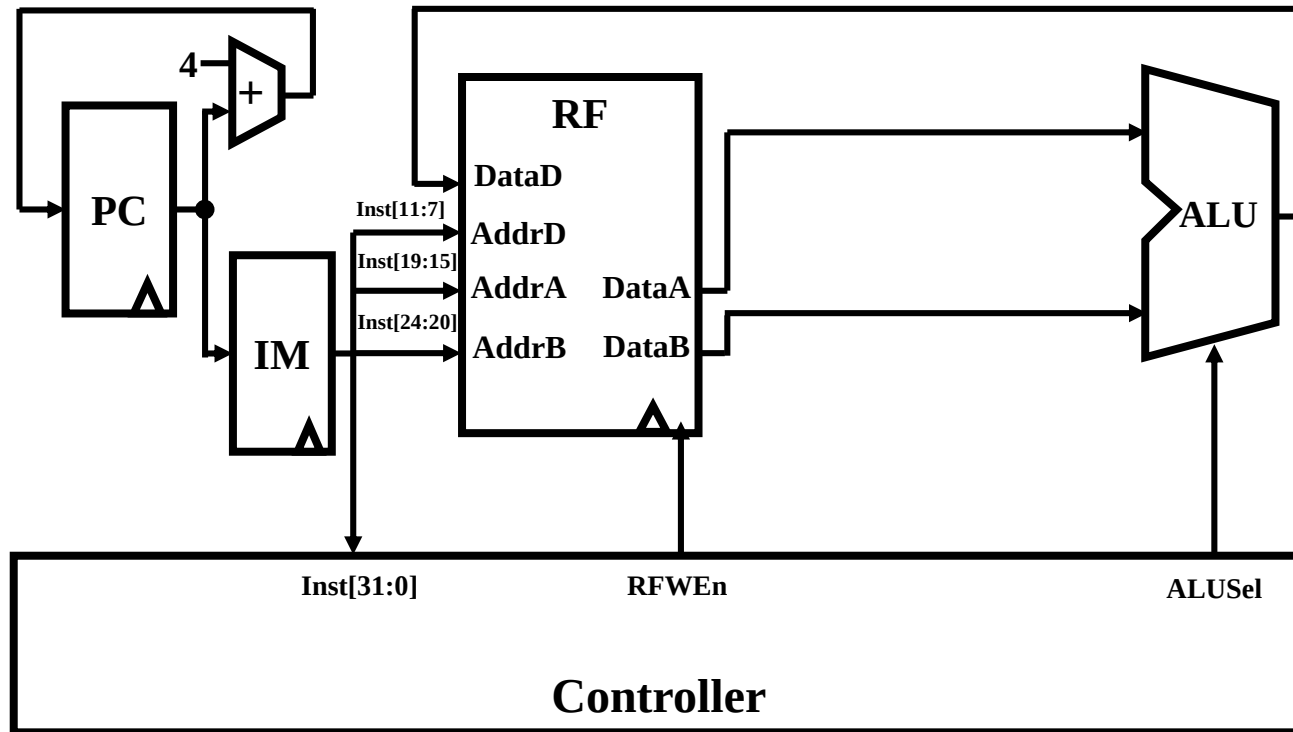
□ addi rd,rs1,imm

□ 指令功能

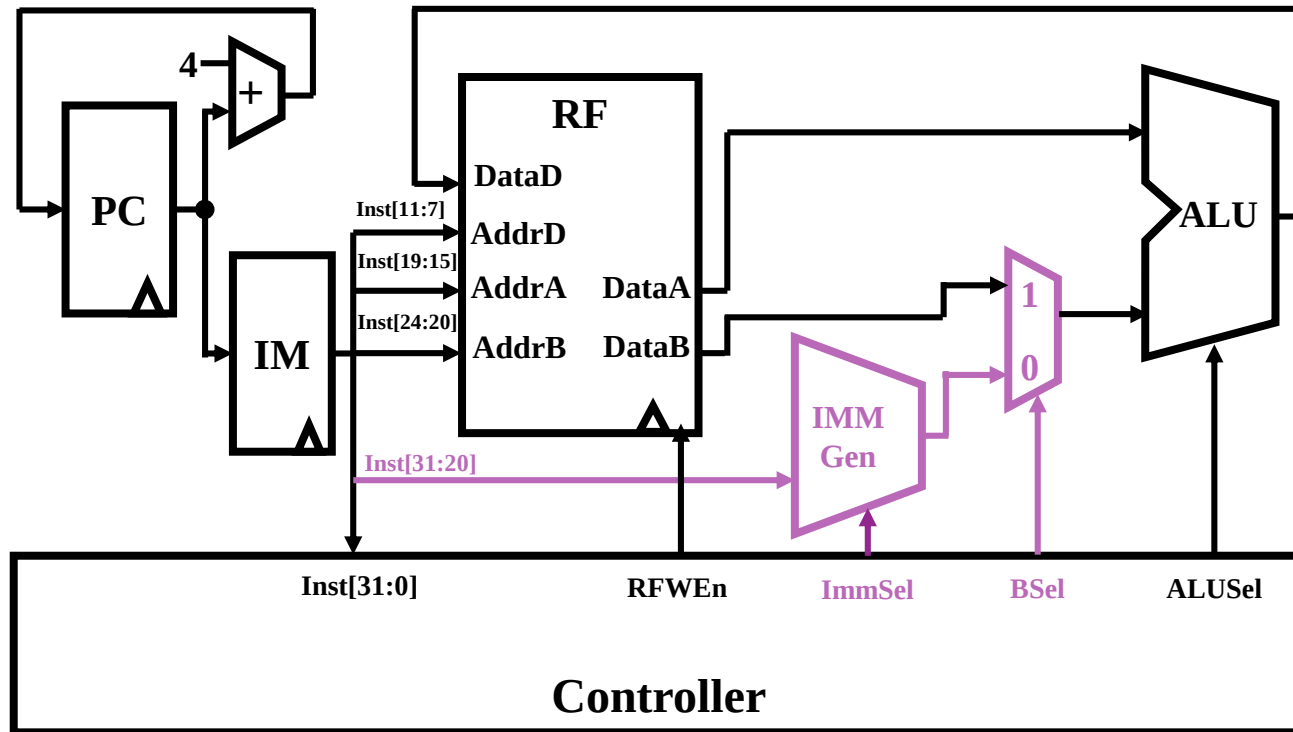
■ $R[rd] = R[rs1] + \text{SignExt}(imm)$



数据通路设计1



数据通路设计2

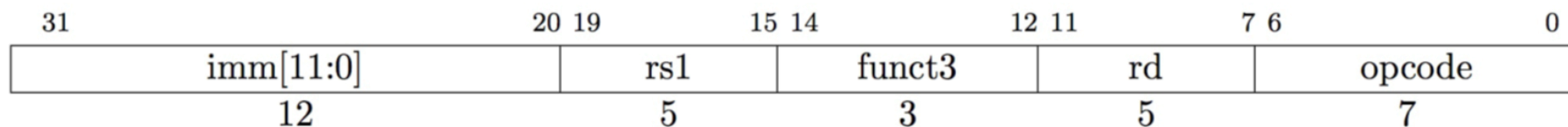


访存I型指令——Load系列

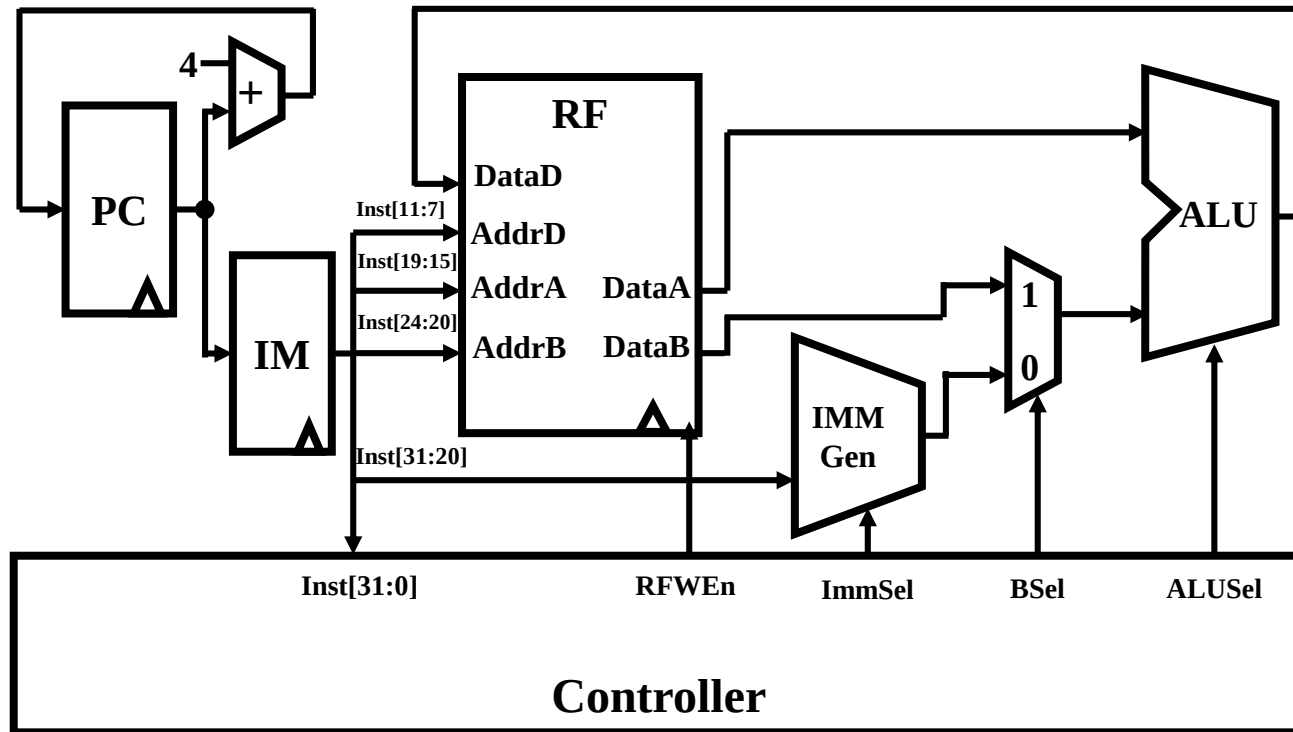
□ Load: lw rd rs1 imm

□ 指令功能

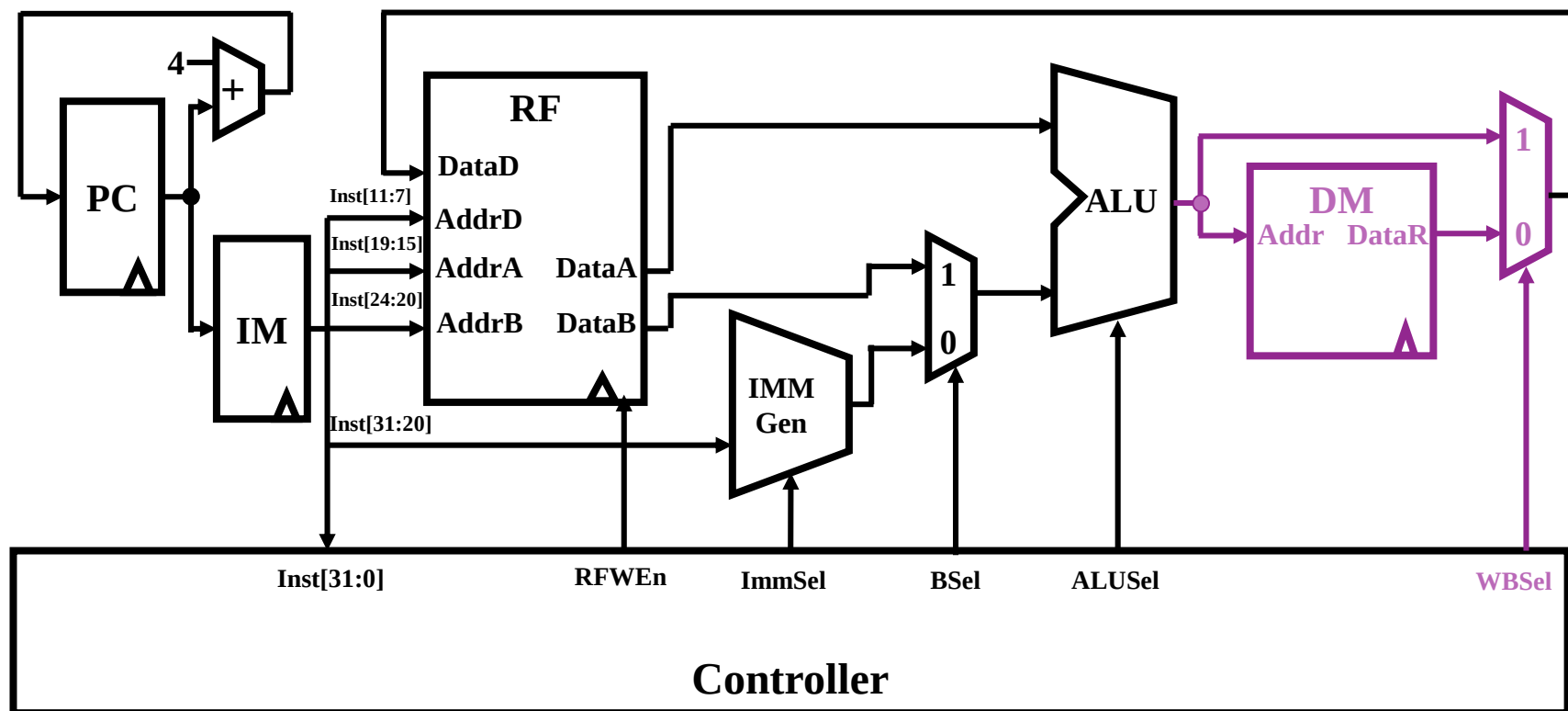
- $\text{Addr} = \text{R}[\text{rs1}] + \text{SignExt}(\text{imm})$
- $\text{R}[\text{rd}] = \text{MEM}[\text{Addr}]$



数据通路设计2



数据通路设计3

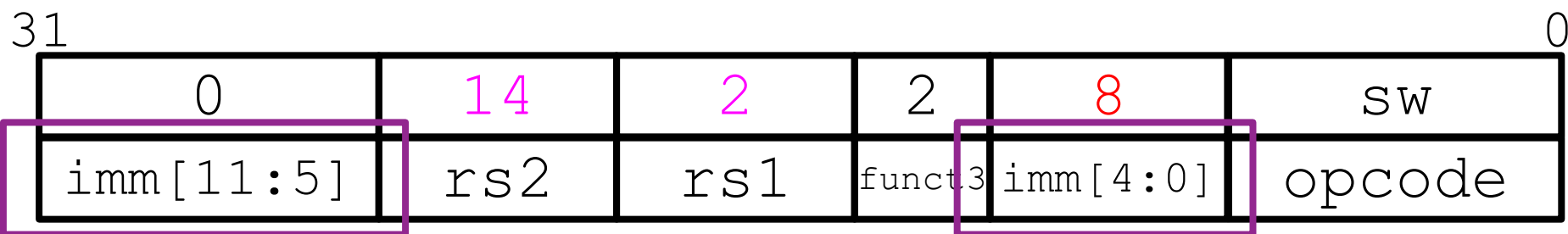


S型指令——Store

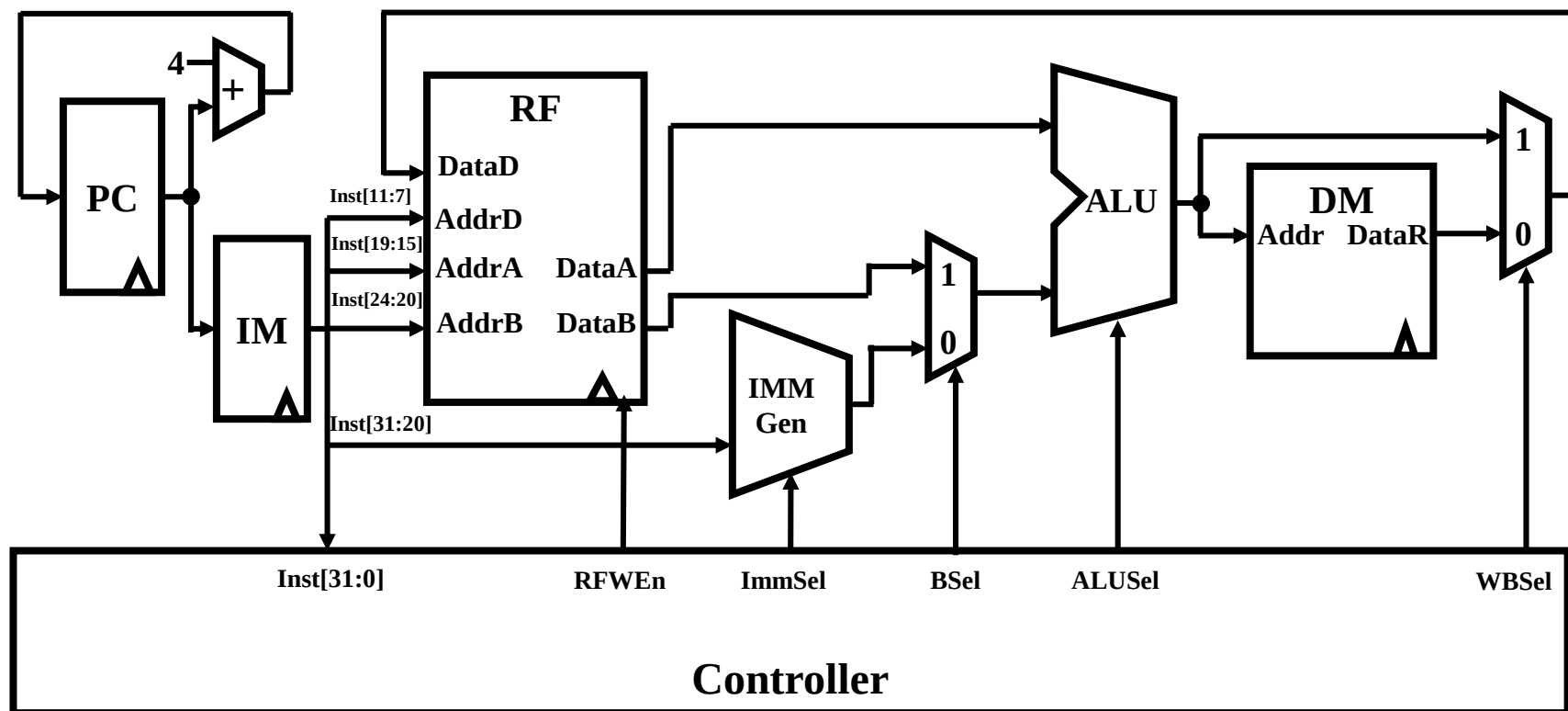
□ Store: sw rs2 rs1 imm

□ 指令功能

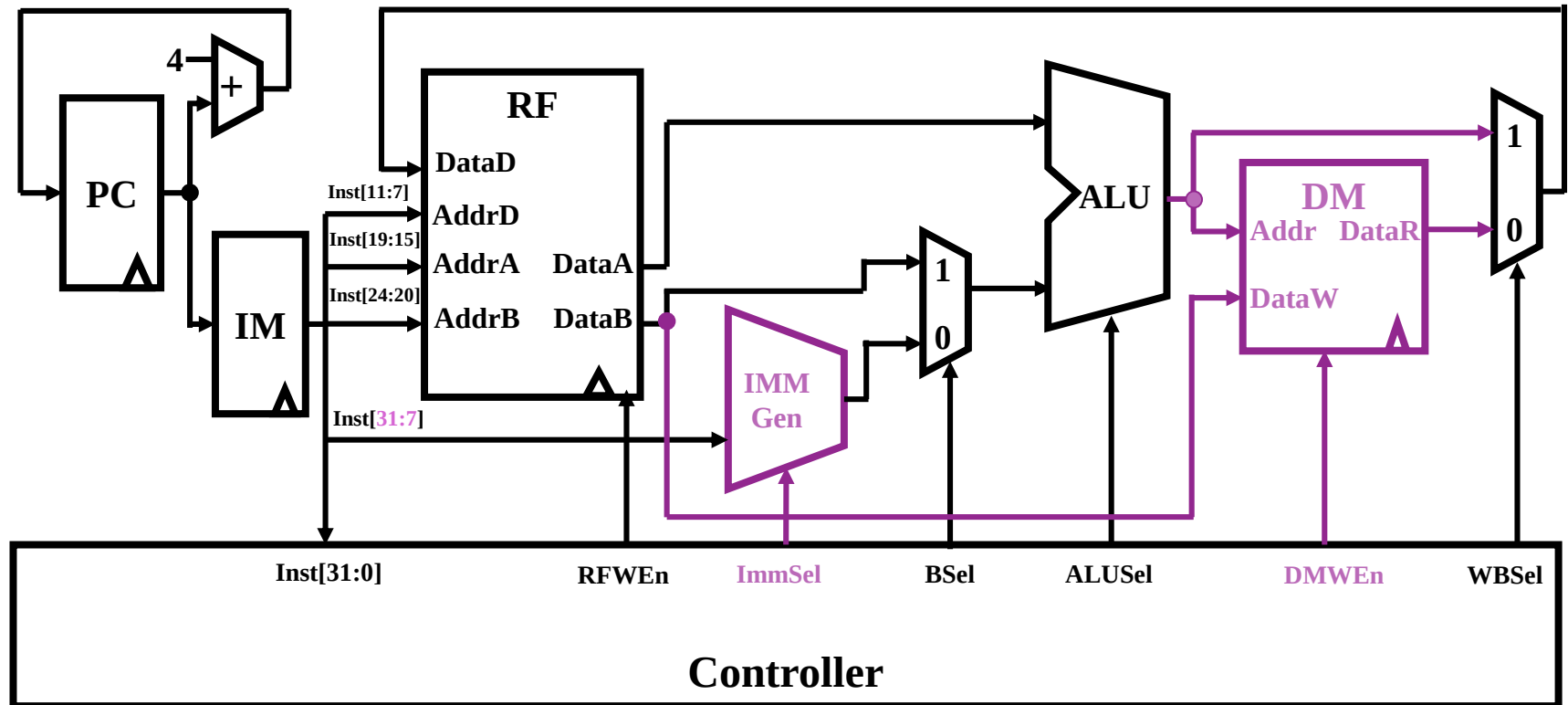
- $\text{Addr} = \text{R}[\text{rs1}] + \text{SignExt}(\text{imm})$
- $\text{MEM}[\text{Addr}] = \text{R}[\text{rs2}]$



数据通路设计3



数据通路设计4

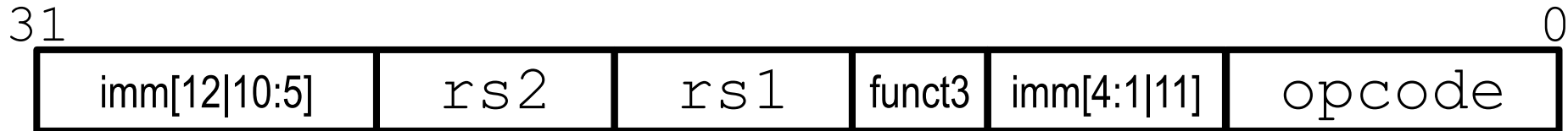


B型指令——BEQ, etc.

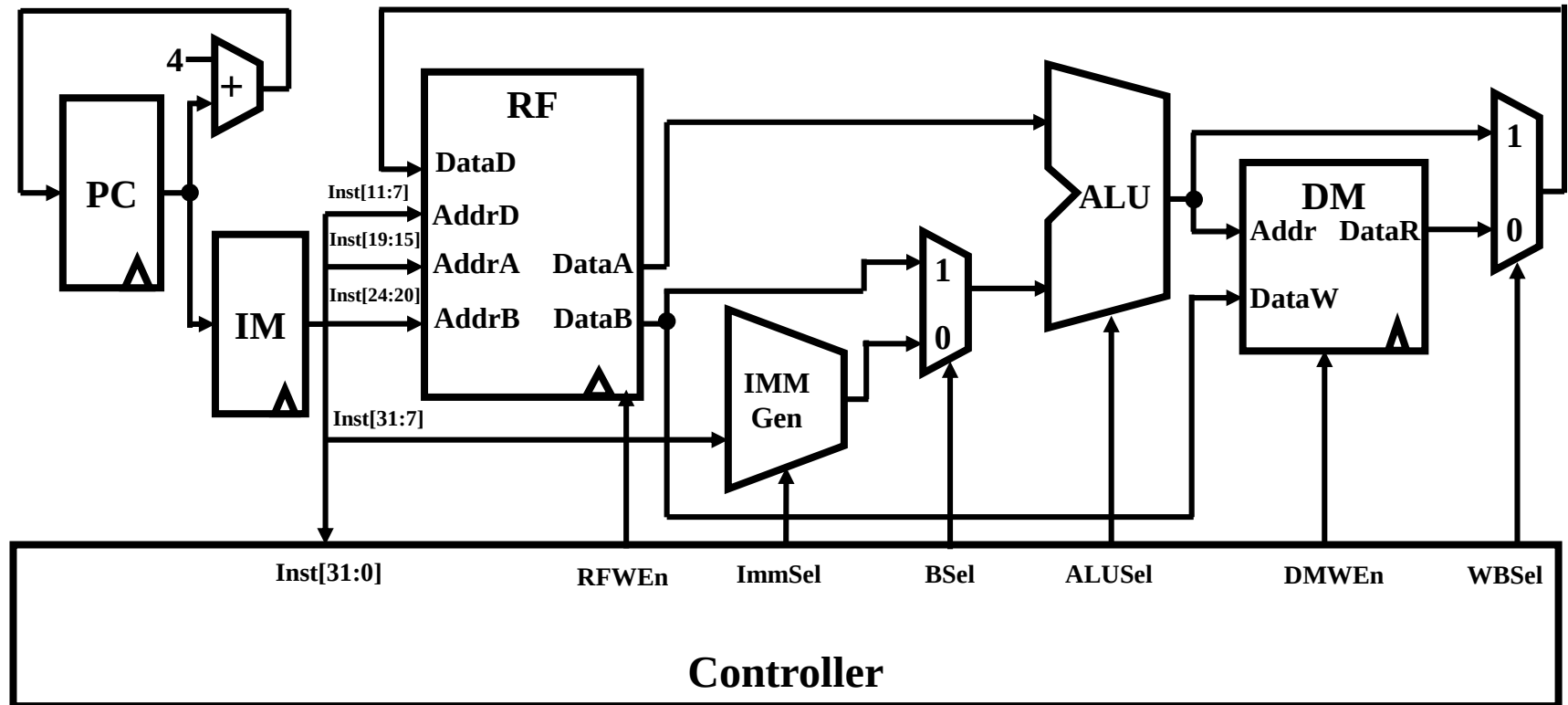
□ BEQ: beq rs1 rs2 label

□ 指令功能

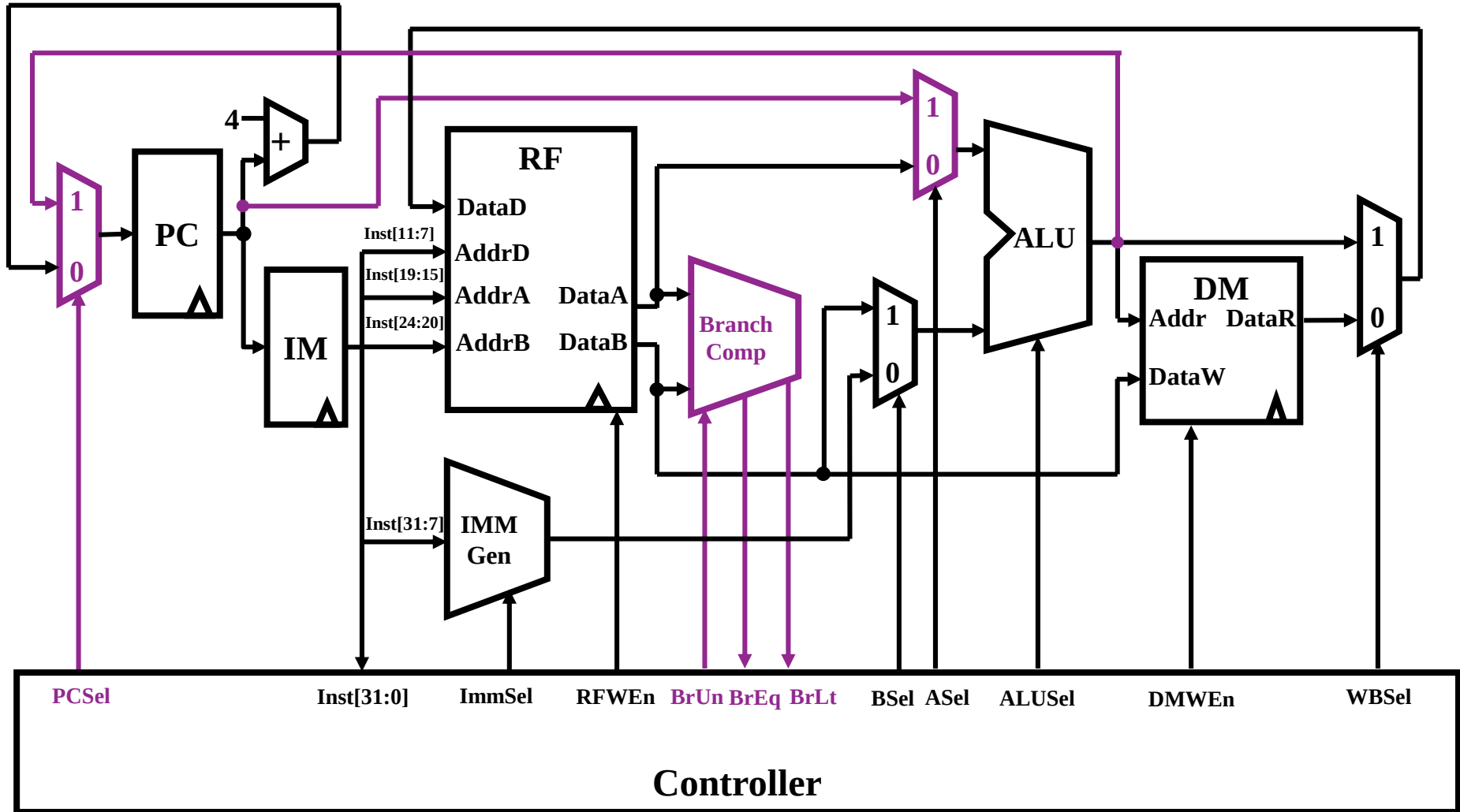
- if $R[rs1] = R[rs2]$
- then $PC \leftarrow PC + \text{SignExt}(imm)$
- Else $PC \leftarrow PC + 4$



数据通路设计4



数据通路设计5



U型指令——lui

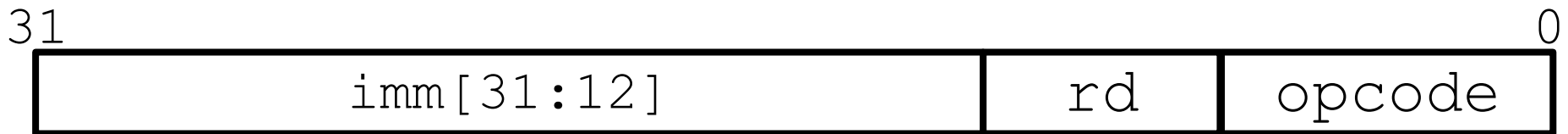
□ auipc reg, imm

□ 指令功能

■ $x[rd] = pc + sext(imm[31:12] \ll 12)$

□ 对数据通路没有提出新的需求

■ 仅立即数生成方式有扩充



J型指令——jal, etc.

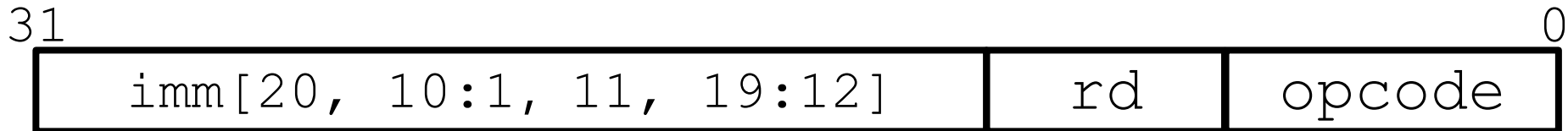
□ Jump: jal rd, imm

□ 指令功能:

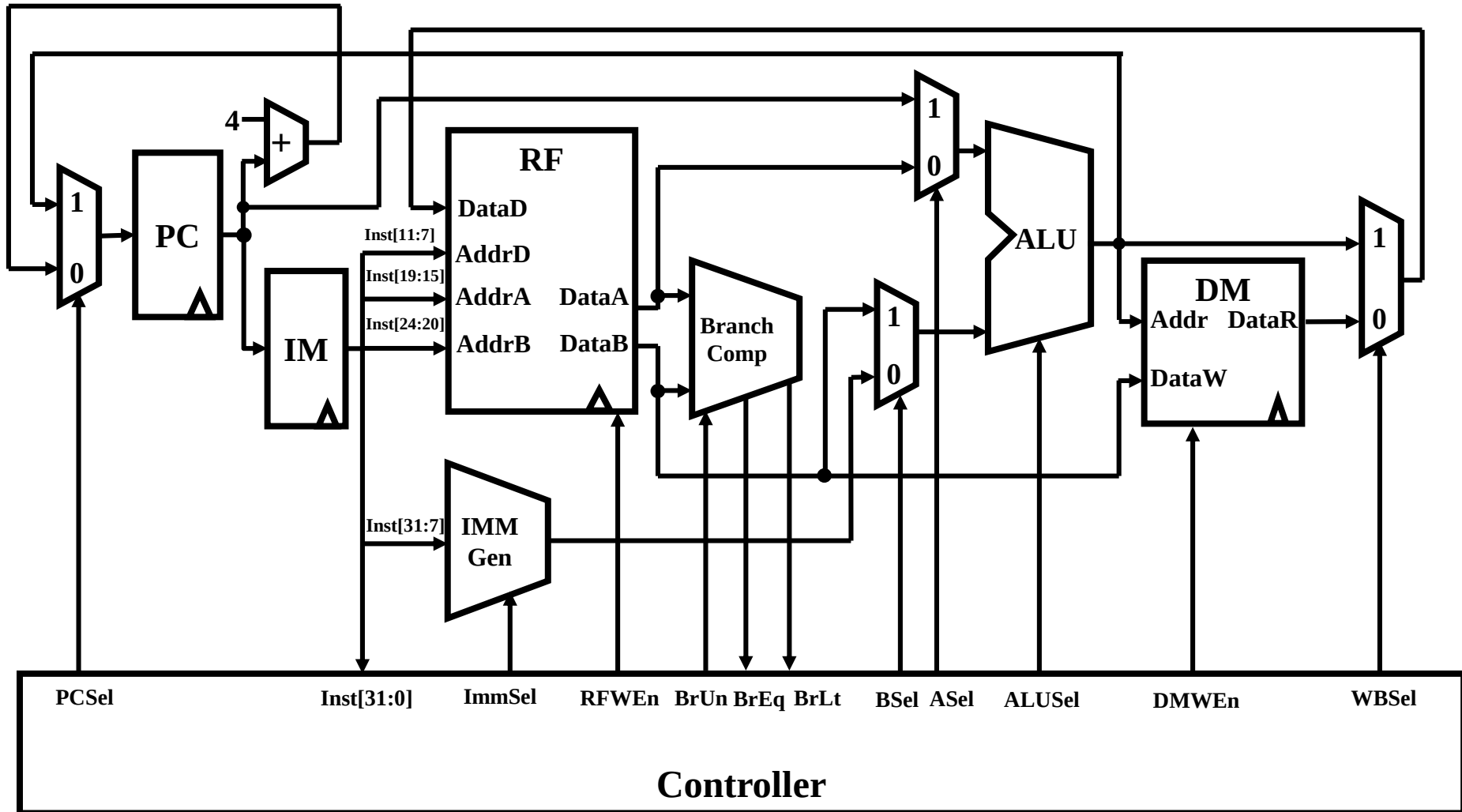
- 置rd值为返回地址 (PC+4)
- 设置跳转 $PC = PC + \text{offset}$

□ 思考: 是否缺少PC+4到RF的数据通路?

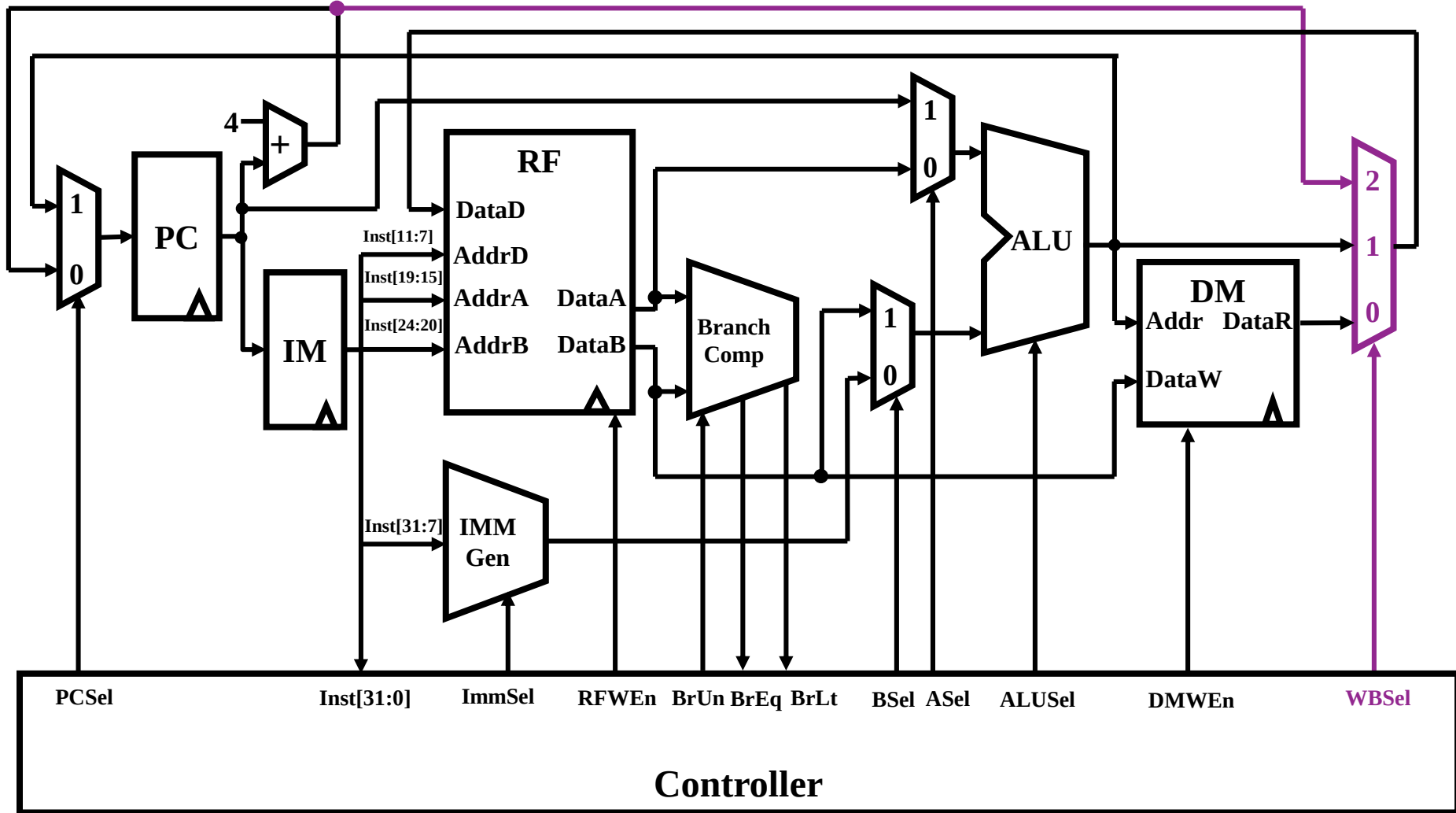
- 修改ALU?
- 新增数据通路?



数据通路设计5



数据通路设计6

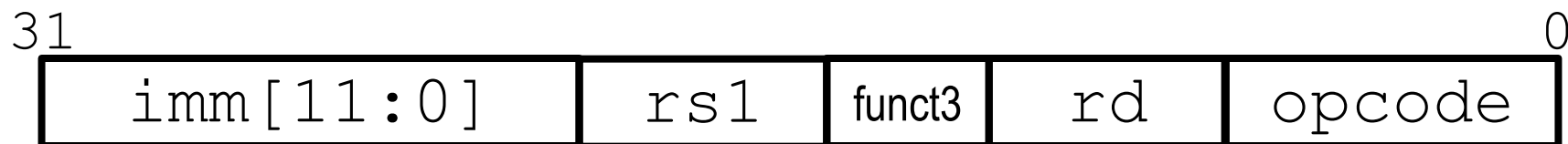


跳转I型指令——jalr, etc.

□ Jalr rd, rs, immediate

- Rd: 返回地址 (PC+4)
- Imm: 偏移offset, 设置跳转 $PC = rs1 + offset$
- Immediate与I类型指令中的算术和加载指令一样

□ 思考：现有数据通路是否可以满足该指令功能？



指令执行过程

- 取指令
- 分析指令
- 读取源操作数
 - 寄存器组
 - 立即数
- 计算
- 访存
- 写回寄存器组

指令功能如何实现？

□ 硬件组成

- ALU
- MEM
- Register File
- ADDer
- PC
- Mux

□ 数据通路

- 如何实现指令的功能？

指令的执行过程与控制

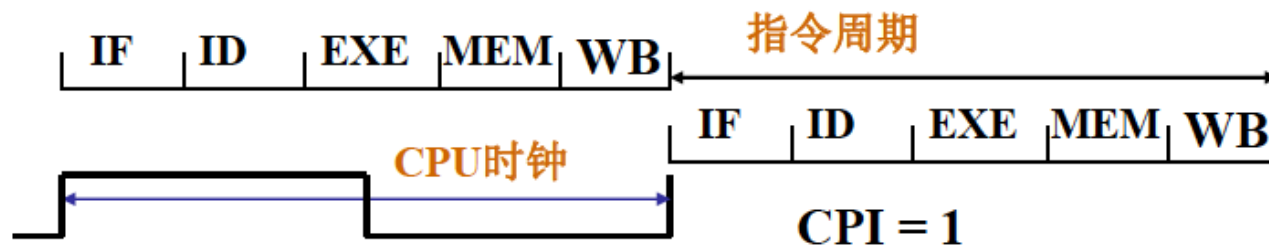
- 冯. 诺依曼结构的计算机，即存储程序计算机，设置内存，存放程序和数据，在程序运行之前存入。
- 执行程序：
 - 正确从程序首地址开始；
 - 正确分步地执行每一条指令，
 - 还要形成下条待执行指令的地址；
 - 并保证自动地连续执行指令，
 - 直到结束程序的最后一条指令。
- 从主存储器读来一条指令，分析指令，按指令的功能要求完成执行过程，本条指令完成后自动开始下一条指令的执行过程，由硬件本身完成。

指令的执行步骤

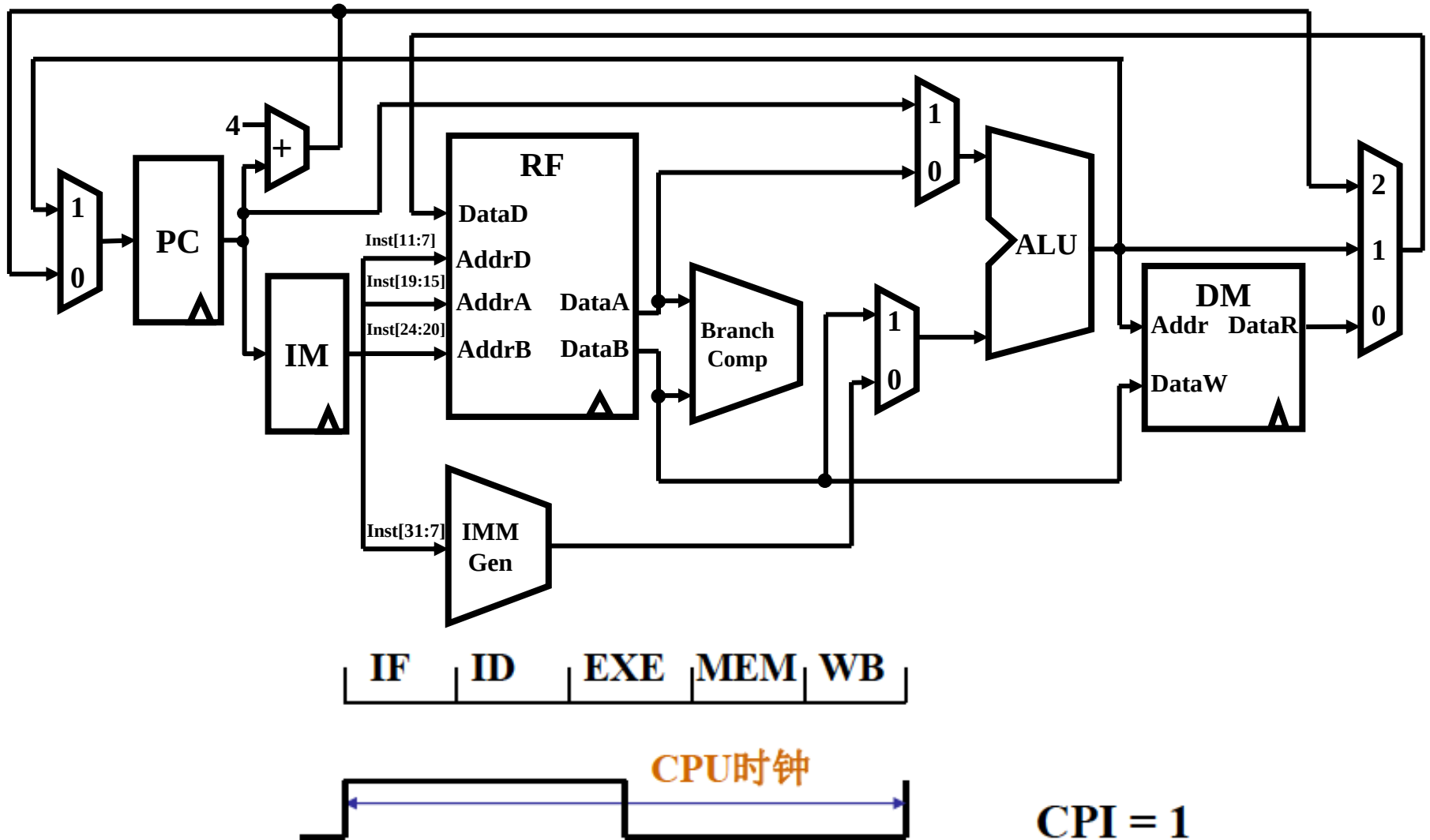
- 当前的计算机系统中，流行的做法是把一条指令的执行过程划分为如下的5 个阶段，是由指令的功能和计算机硬件结构共同决定的，有内在的逻辑关系。
 - 读取指令(IF), 从存储器读来指令并形成下条指令地址
 - 指令译码(ID), 指令译码，读寄存器堆为ALU准备数据
 - 执行运算(EXE), ALU 执行数据运算或计算存储器地址
 - 存储器读写(MEM), 完成存储器的读操作或者写操作
 - 写回(WB), 写ALU的结果或存储器读出数据到寄存器堆
- 从如何为不同指令安排这几个阶段，几个阶段如何衔接考虑，大体有3 种可行方案，各自都对计算机的内部结构、部件设置及其控制方式有着不同要求。

单周期CPU

- ❑ 计算机一条指令的执行时间被称为**指令周期**，一个CPU时钟时间被称为**CPU周期**(在某些计算机中，还可再把一个CPU周期区分为几个更小的步骤，称其为节拍)。执行**每条指令平均使用的CPU周期个数**被称为**CPI**
- ❑ 全部指令都选用**一个CPU周期**完成的系统被称为**单周期CPU**，指令串行执行，前一条指令结束后才启动下条指令。每条指令都用5个步骤的时间完成，控制各部件运行的信号在整个指令周期不变化。**单周期CPU**用于早期计算机，系统性能和资源利用率很低，相对当前技术变得**不再实用**。

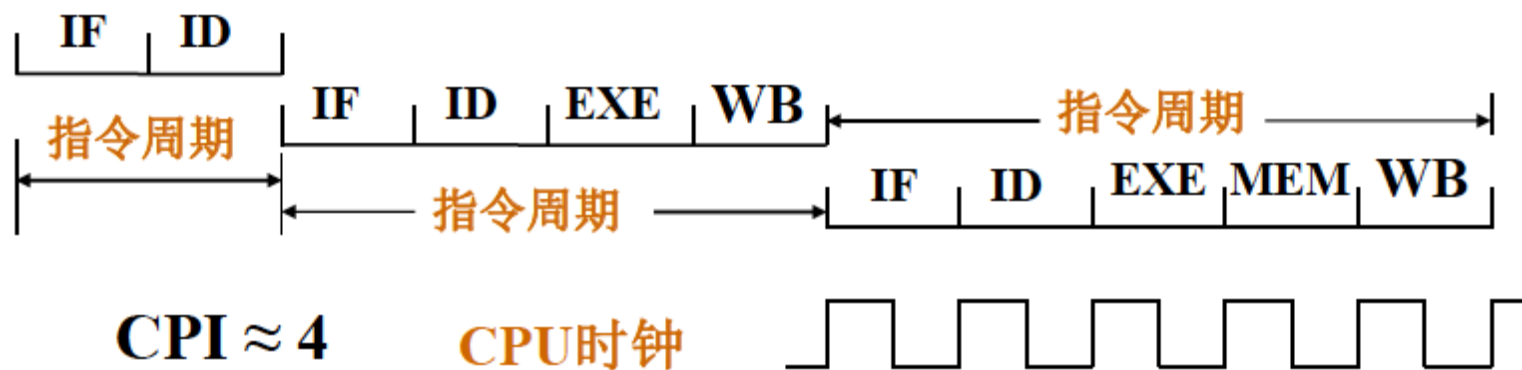


单周期CPU

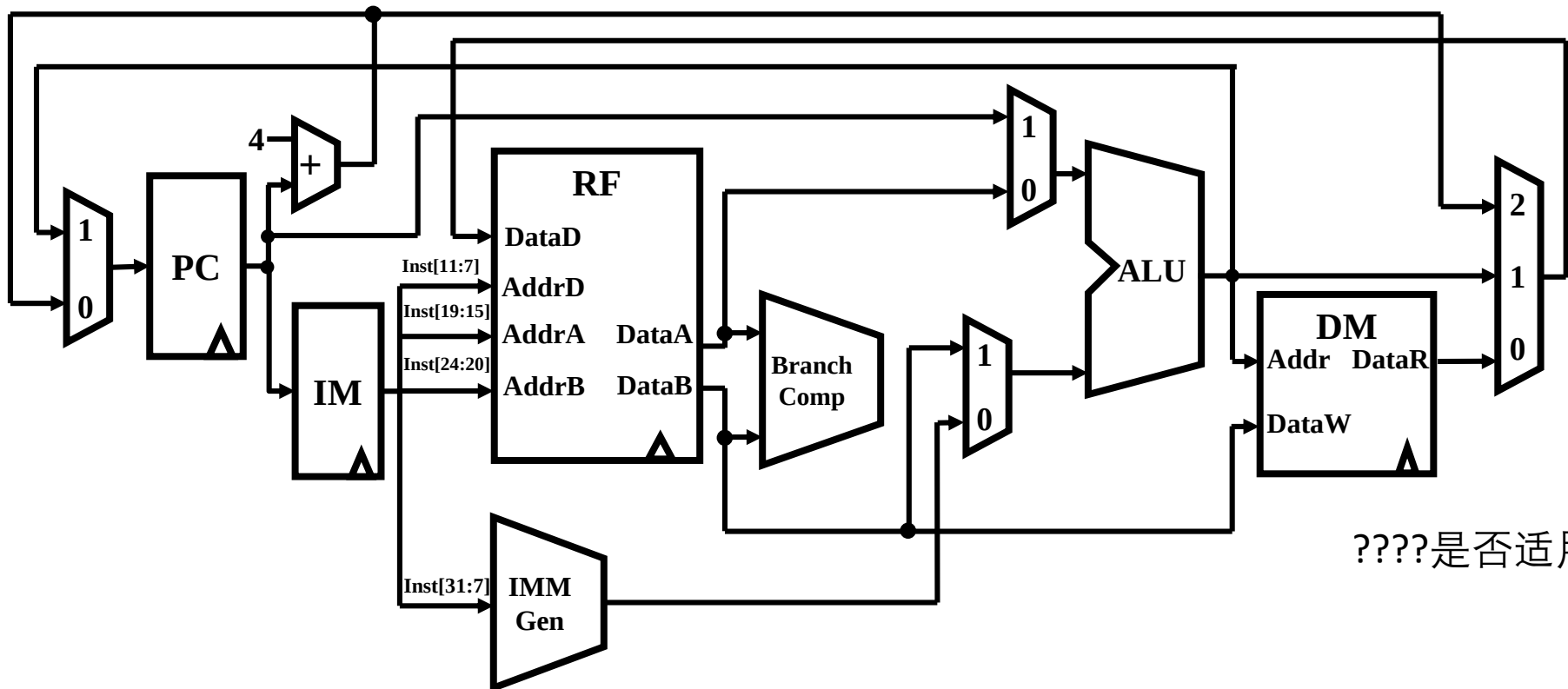


多周期CPU

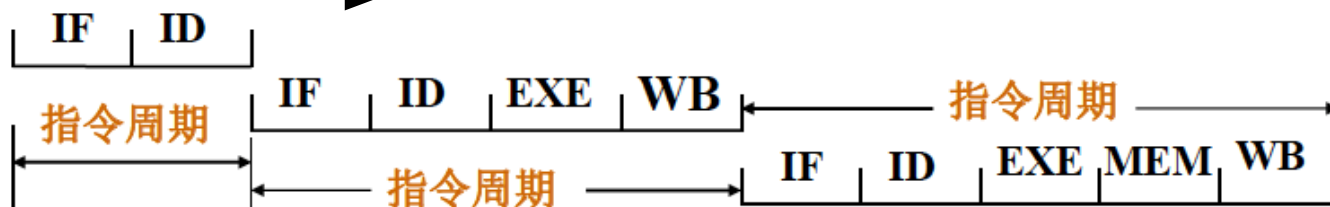
- ❑ 计算机一条指令的执行时间被称为指令周期，一个CPU时钟时间被称为CPU周期。
- ❑ 依据不同指令各自的功能需求为其选择不等的执行步骤的系统被称为多周期CPU，控制各部件运行的控制信号随着指令执行步骤改变，系统性能和资源利用率更高。相邻指令可以完全串行执行，也可能部分时间重叠，多周期CPU（相比单周期CPU）更实用。



多周期CPU



????是否适用?



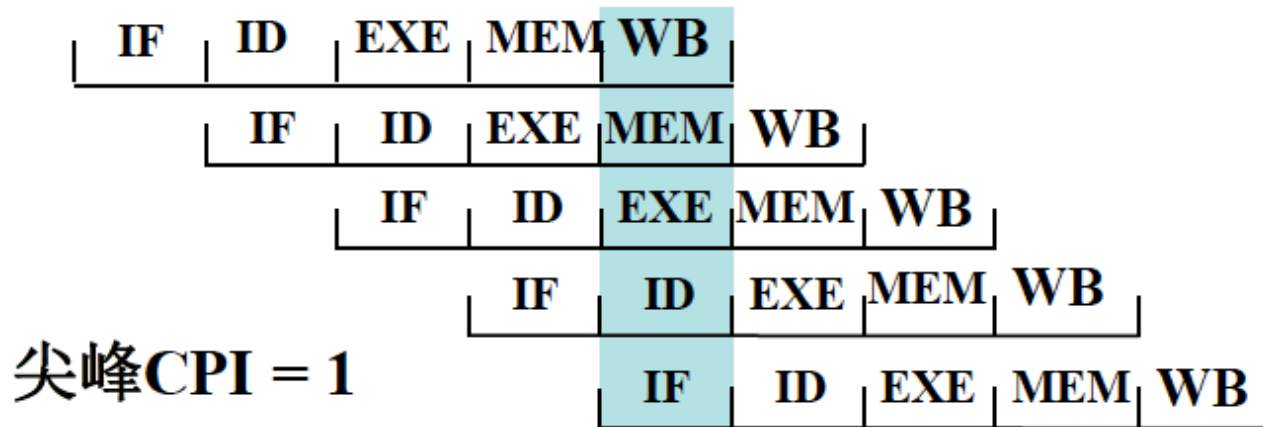
CPI ≈ 4

CPU时钟



指令流水线CPU

- 全部指令都是选用5个步骤完成，执行时间相同,但相邻指令的执行并不是完全串行的，执行时间有所重叠，例如每结束指令的一个执行步骤就启动下条指令，这被称为**指令流水线技术**，所有部件都高速运行，尖峰速度每个CPU时钟执行一条指令，系统性能和资源利用率更高，显著地提高系统的性能价格比，但**计算机结构和控制器的设计、实现略显复杂**。**当前计算机中普遍使用这种方案**。



小结

- 数据通路是指令功能实现的关键
- 寻址方式是影响数据通路设计的重要因素
- RISC，以简洁换取性能提高
- CISC，以丰富换取编程方便
- 指令执行
 - 单步骤串行
 - 多步骤串行
 - 流水

阅读和思考

□ 阅读

□ 思考

□ 实践

- 分析ThinPAD RV指令系统的特点，并对其指令格式进行分类
- 设计能实现ThinPAD RV指令系统的数据通路，尤其要注意专用寄存器的实现方法

谢谢